Proiectare Logică

I CA & CB: Mariana Mocanu
mariana.mocanu@cs.pub.ro
I CC & CD: Costin Chiru costin.chiru@cs.pub.ro

Curs 10: Structuri microprogramate.

Introducere

- Îmbină algoritmii de la programare cu circuitele învățate deja.
- Ce am făcut până acum (ASM-uri) se pretează la un număr mic de stări (ce pot fi implementate folosind maxim 4-5-6 variabile de stare).
- Structurile microprogramate sunt folosite pentru aparate cu un număr mai mare de stări.
- Partea de logică a structurii microprogramate este conținută într-un microprogram

Recapitulare (1)

- La circuitele secvențiale avem 2 elemente importante:
  1. **Stările circuitului**
     - Caracterizate
       - Printr-o configurație unică a variabilelor de stare;
       - De condițiile ce au făcut să ajung în starea curentă plecând din starea anterioară;
       - De ieșirile de la momentul curent de timp.
     - Au o anumită durată (în funcție de ceas la circuitele sincrone sau de durata dintre 2 tranziții la circuitele asincrone).
     - Memorează un anumit parcurs al stărilor (ex: ce biți au fost citiți deja)

Recapitulare (2)

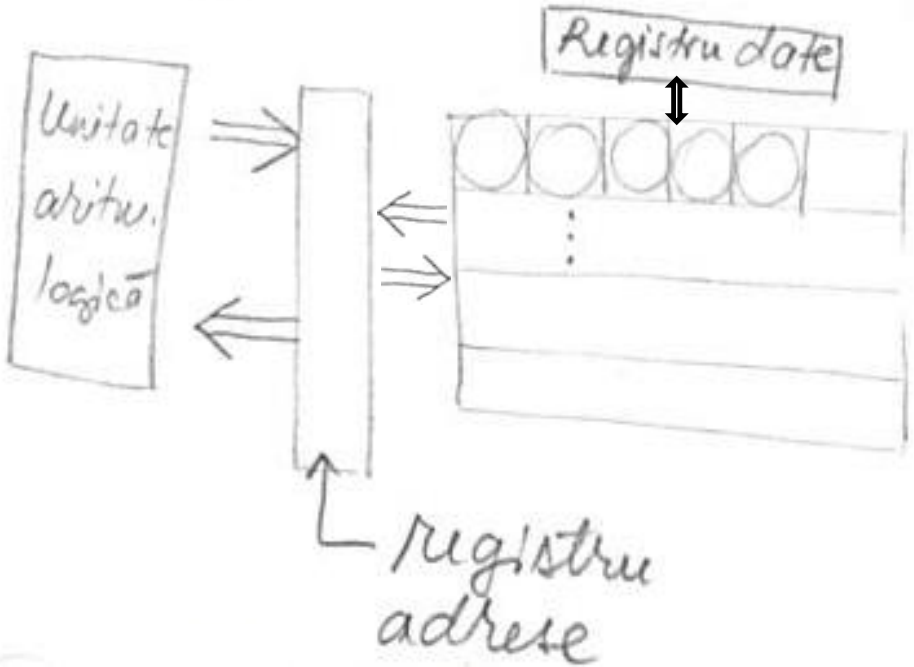
- La circuitele secvențiale avem 2 elemente importante:
  2. **Tranzițiile dintre stări**
     - Pot fi de 2 feluri:
       - Condiționate de variabilele de intrare
       - Necondiționate: se trece direct în starea următoare
     - Sunt considerate a fi instantanee

Memoria

- Ce știm despre ea?
- Nu ne-am ocupat explicit de memorii, dar am folosit elemente de memorare
- Unde am întâlnit?
- La circuitele secvențiale: orice circuit secvențial are un element de memorare
- Unde am mai vorbit despre memorii?
- 4 CBB-uri legate între ele pot reprezenta un cuvânt de memorie (de 4 biți)
- Lărgim spectrul – ce știți despre memorie? Unde ați mai întâlnit?
- Programare – stive; alocarea din C

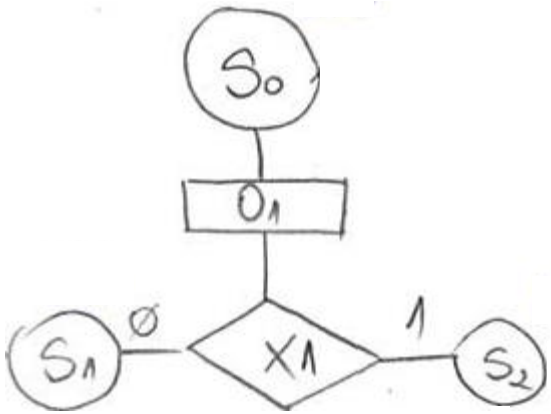
Memoria

- Un circuit de memorie este alcătuit din locații de memorie
- Cum pot să ajung la una sau alta din locațiile de memorie?
- Folosind registrul de adrese RA (orice memorie trebuie să aibă structural un RA)
- Ce facem cu locația de memorie?
- O punem în registrul de date RD unde salvăm / de unde preluăm informații



Memoria

- Ex: pentru un cuvânt (o locație) de memorie de 8 biți putem avea 8 CBB de tip D, pentru următoarea alte 8 CBB-uri etc.
- În realitate, circuitele sunt LSI sau VLSI și nu mai putem să vedem individual fiecare circuit, dar în principiu fiecare bit este realizat printr-un circuit secvențial
- În fiecare stare dintr-un aparat este memorată informația despre care e starea următoare și eventual despre ieșirile din starea curentă



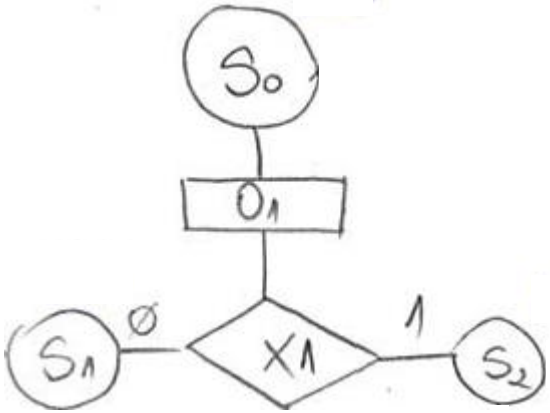
Structuri microprogramate

- Structurile microprogramate merg pe ideea că într-o organigramă avem o succesiune de stări și informația asociată fiecărei stări în parte o memorăm într-o microinstrucțiune
- **Microinstrucțiune** = toate informațiile legate de stare, ce se memorează într-o memorie cu caracter permanent
- Fiecare microinstrucțiune se găsește la o anumită adresă în memorie
- Ansamblul microinstrucțiunilor formează **microprogramul** și se memorează ca o succesiune de microinstrucțiuni

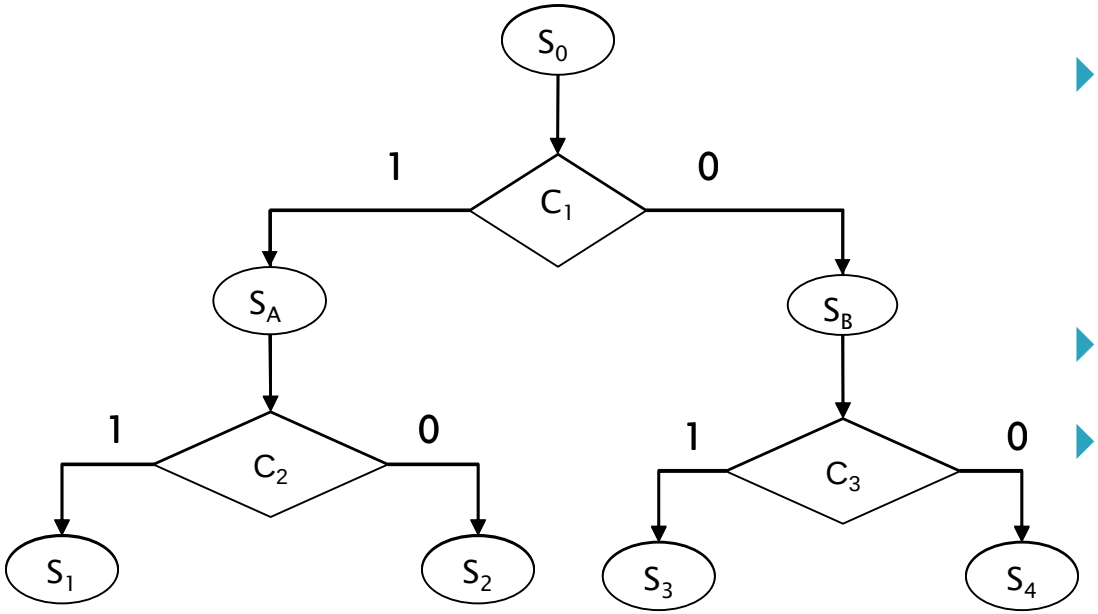
Memorie necesară

- Ex: dacă avem 3 variabile de stare → 8 stări → avem nevoie de o memorie de 8 cuvinte
- dacă avem un ASM cu 10 variabile de stare → 2^{10} = 1024 stări → avem nevoie de o memorie de 1024 de cuvinte (locații)
- Nu știm ce înseamnă o microinstrucțiune, dar știm că fiecare stare are asociată o microinstrucțiune → similar cu liniile dintr-un program (instrucțiuni): fiecare instrucțiune spune ce trebuie să se execute și dacă se trece la următoarea instrucțiune sau sărim la o alta în urma unor decizii (de ex. if).

Microinstrucțiuni



- Structura tipică este
- Ce se întâmplă dacă avem condiții complexe?

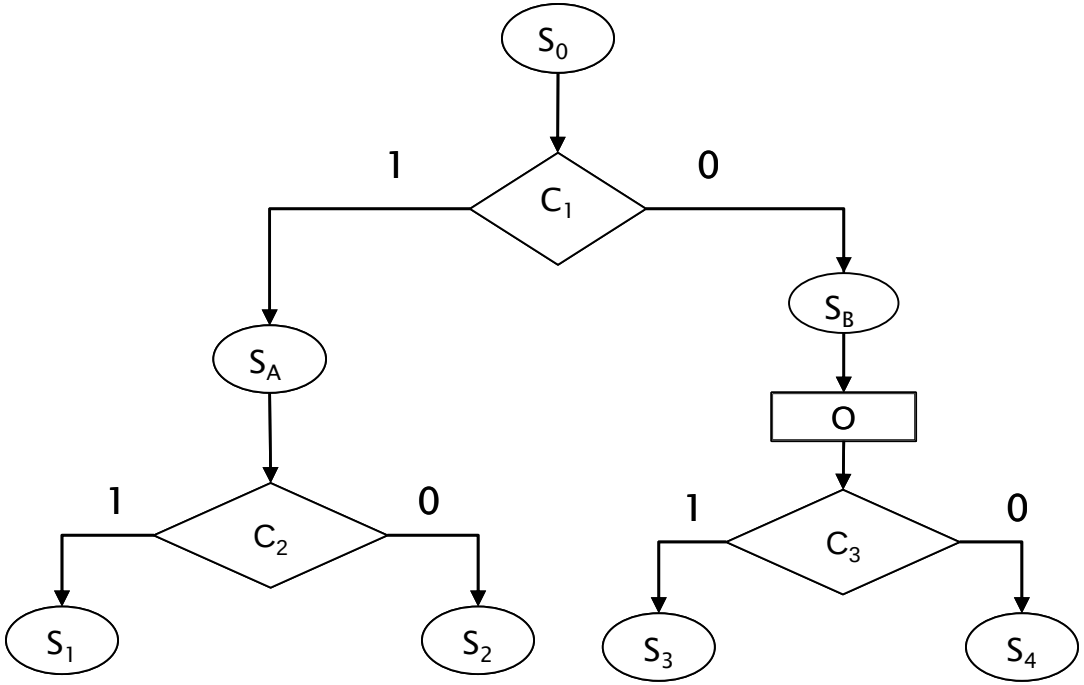


- Aducem organigrama la structura clasică
- Cum?
- Introducând stări noi

Microinstrucțiuni

- Prima diferență față de ASM:
  - La ASM trebuia să facem organigrama cât mai compactă pentru a limita numărul de stări → variabile de stare → Diagrame Karnaugh cât mai mici
  - La structuri microprogramate se lucrează de obicei cu aparate de tip Moore (se poate și cu Mealy dar e mai complicat → se pune mască pe ieșiri și se complică schema). Drept urmare, trebuie să modific schema pentru a transforma zonele de tip Mealy în zone de tip Moore

Mealy – Moore

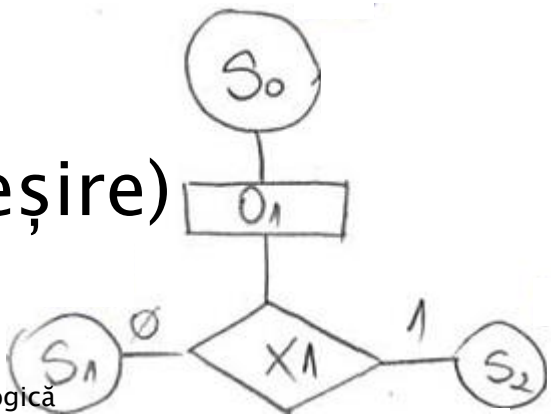


Structura microinstrucțiunii

- O microinstrucțiune are mai multe câmpuri pentru a reține informațiile legate de semnalul de intrare care cauzează tranziția, adresele stărilor următoare si ieșirile generate:

Input Code	ADR 0	ADR 1	OUT

- O parte din informații le scriem codificat, altele le scriem decodificat
- Intrările și stările → codificate
- Ieșirile → decodificate (un bit/ieșire)

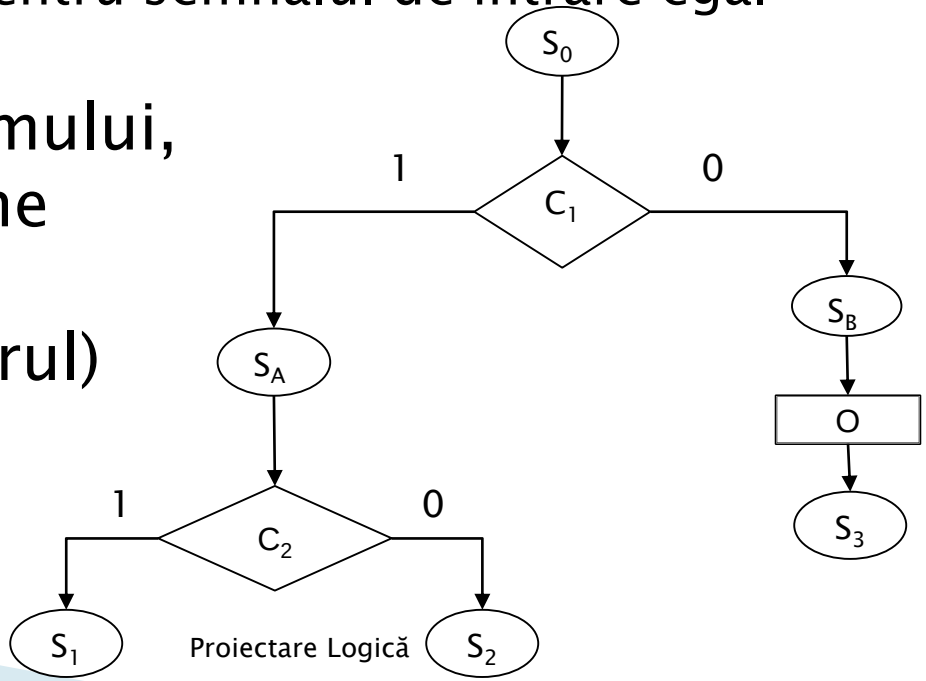


Input code

- Input Code (INPCOD) = codul semnalelor de intrare
- Cum se obține codul?
- Ex: avem 3 intrări: I_1, I_2, I_3 și încerc să le dau coduri binare. Câți biți sunt necesari?
- 2: I_1 – 01, I_2 – 10, I_3 – 11 (codificăm la rând și dacă se poate lăsăm combinația formată doar din 0 nefolosită)
- De ce codificăm? Ca să câștigăm biți (reprezentare pe 2 biți în loc de 3)
- Ex2: avem 14 intrări. Câți biți sunt necesari?
- 4 biți → economie de 10 biți

Starea următoare

- Presupunem că avem ceva de genul:
- Din S_B mă duc în S_3, dar din S_0 și S_A mă pot duce în 2 stări și atunci:
  - ADR0 = adresa următoare pentru semnalul de intrare egal cu 0
  - ADR1 = adresa următoare pentru semnalul de intrare egal cu 1
- În memoria microprogramului, pentru fiecare instrucțiune aloc un cuvânt (nu e obligatoriu să fie următorul) → vom vedea mai târziu



Starea următoare

- Exemplu: memorie de 8 cuvinte (3 variabile de stare). Câți biți conține adresa?
- 3
- Ce face adresa?
- Selectează una din instrucțiuni (stări) → e un cod
- Cum știu unde să mă duc mai departe?
- Trebuie să selectez adresa pe care merg
- Ex: dacă sunt în S_0, trebuie să am un aparat la care dacă condiția e 1, RA să genereze codul lui S_A, iar dacă e 0 codul lui S_B

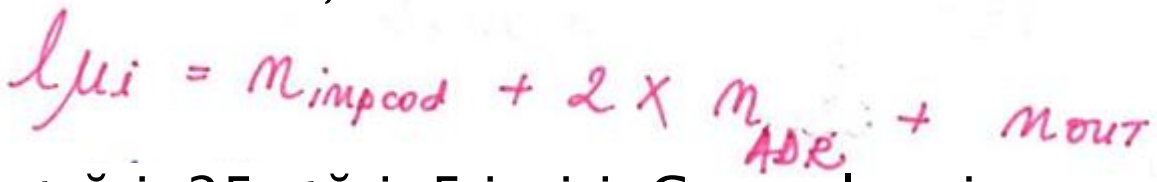


Ieșiri

- OUT = semnalele de ieșire generate în starea curentă
- Nu se folosesc codificări pentru că în fiecare stare pot genera mai multe ieșiri și atunci pot avea ceva de forma:
  - S_0: O1, O4
  - S_1: O1, O2
- Nu se câștigă nimic codificându-le pentru că trebuie să se țină cont și de posibilele combinații de ieșiri → reprezentare fiecare ieșire printr-un bit 0/1

Lungimea microinstrucțiunii

- Având forma microinstrucțiunii, trebuie să calculăm lungimea acesteia, folosind formula:



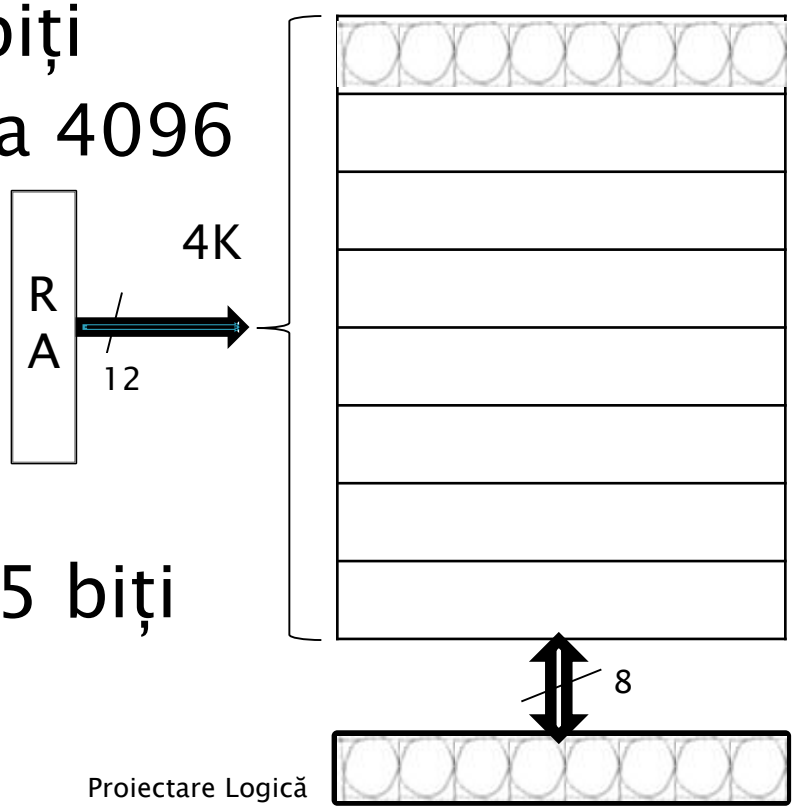
- Ex: 7 intrări, 25 stări, 5 ieșiri. Care e lungimea microinstrucțiunii?
- Câți biți sunt necesari pentru codificarea intrărilor?
- Câți biți sunt necesari pentru codificarea stărilor?
- Fiecare stare = 1 cuvânt din memorie → avem 25 de cuvinte în memorie → 5 biți necesari pentru reprezentare → 5 biți pentru RA
- $L_{\mu i} = 3 + 5 + 5 + 5 = 18$ biți
- Teoretic am nevoie de o memorie cu RA de 5 biți, RD de 18 și capacitate de 25 de cuvinte a câte 18 biți fiecare
- În realitate, memoriile sunt standardizate → nu pot avea orice număr de biți pentru RA & RD & capacitate memorie

Problemă (1)

- Fie un circuit de memorie de 4K * 8 biți. Calculați lungimea microinstrucțiunii și stabiliți câte astfel de circuite de memorie sunt necesare dacă se știe că avem 37 de semnale de intrare și 25 de semnale de ieșire.
- Ce știm?
- Memoria are 4K * 8 (lungimea cuvântului)
- Ce lipsește?
- Numărul de stări.
- Care e numărul maxim de stări?

Problemă (2)

- Câte stări poate avea microprogramul?
- Maxim 4K adrese în memorie = maxim 2^{12} = 4096 stări posibile → numărul maxim de stări
- Pentru adresarea lor: 12 biți
- RA → 12 biți → pot adresa 4096 poziții → aparatul poate avea maxim 4096 stări
- $L_{\mu i} = n_{ci} + 2 * n_{adr} + n_{out}$
- 37 intrări, 25 ieșiri
- $L_{\mu i} = 6 + 2 * 12 + 25 = 55$ biți



Problemă (3)

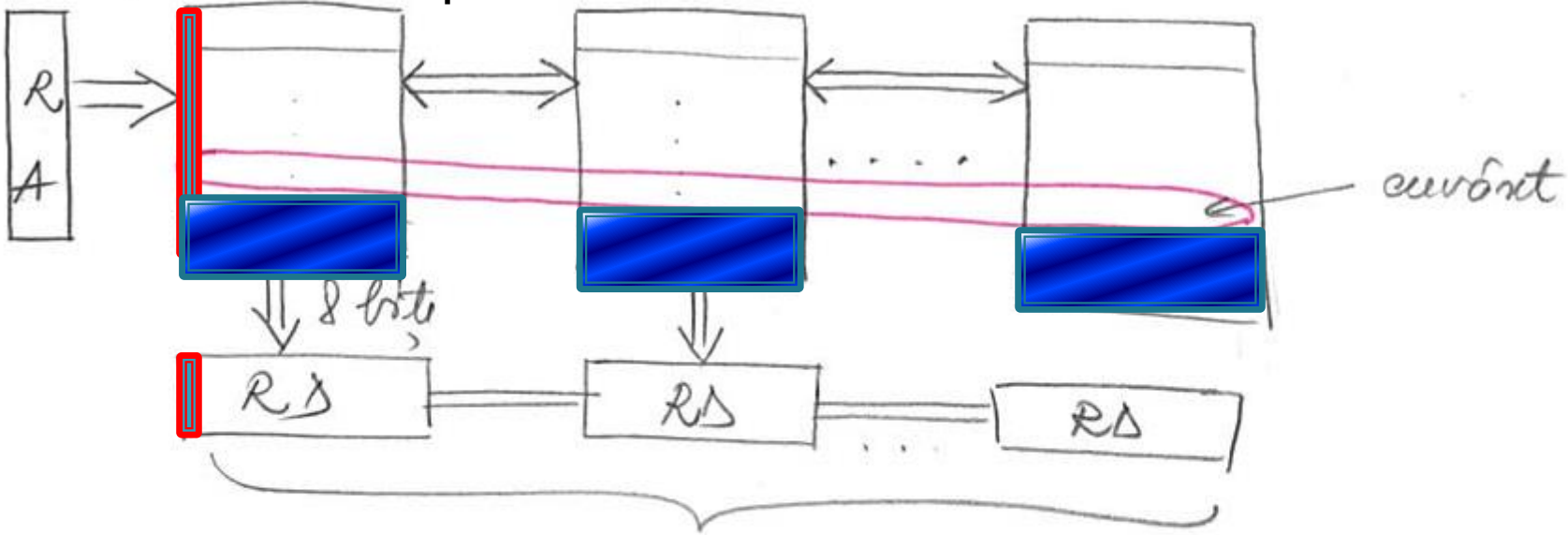
- Pot folosi această memorie pentru un aparat cu 16 stări?
- Da
- Se modifică lungimea microinstrucțiunii?
- Nu, pentru că ele sunt adresate în continuare folosind RA care trebuie să poată să numere pâna la 4095 (chiar dacă majoritatea locațiilor vor fi libere).

Problemă (4)

- De unde vine întrebarea a doua: de câte circuite avem nevoie?
- $L_{\mu i} = 55$ biți, dar în memorie eu am cuvinte de maxim 8 biți → ce facem?
- Luăm mai multe astfel de circuite și le cuplăm în paralel (le „concatenăm”)
- Un cuvânt este reprezentat de concatenarea RD-urilor
- Câte circuite sunt necesare? $\lceil 55/8 \rceil = 7$ (1 bit din RD cel mai semnificativ rămâne nefolosit)
- Toate RD sunt comandate simultan de același RA și datele sunt selectate și copiate simultan în RD – uri. Concatenarea lor conduce la cuvântul microinstrucțiunii

Problemă (5)

- Dacă aparatul are mai puține stări (nu 4096)?
- Există o zonă de memorie nefolosită în fiecare dintre circuite
- Dacă $L_{\mu i}$ nu este multiplu al dimensiunii cuvântului memoriei, rămâne o zonă de memorie nefolosită în memoria corespunzătoare MSB



55 biți

7 capsule

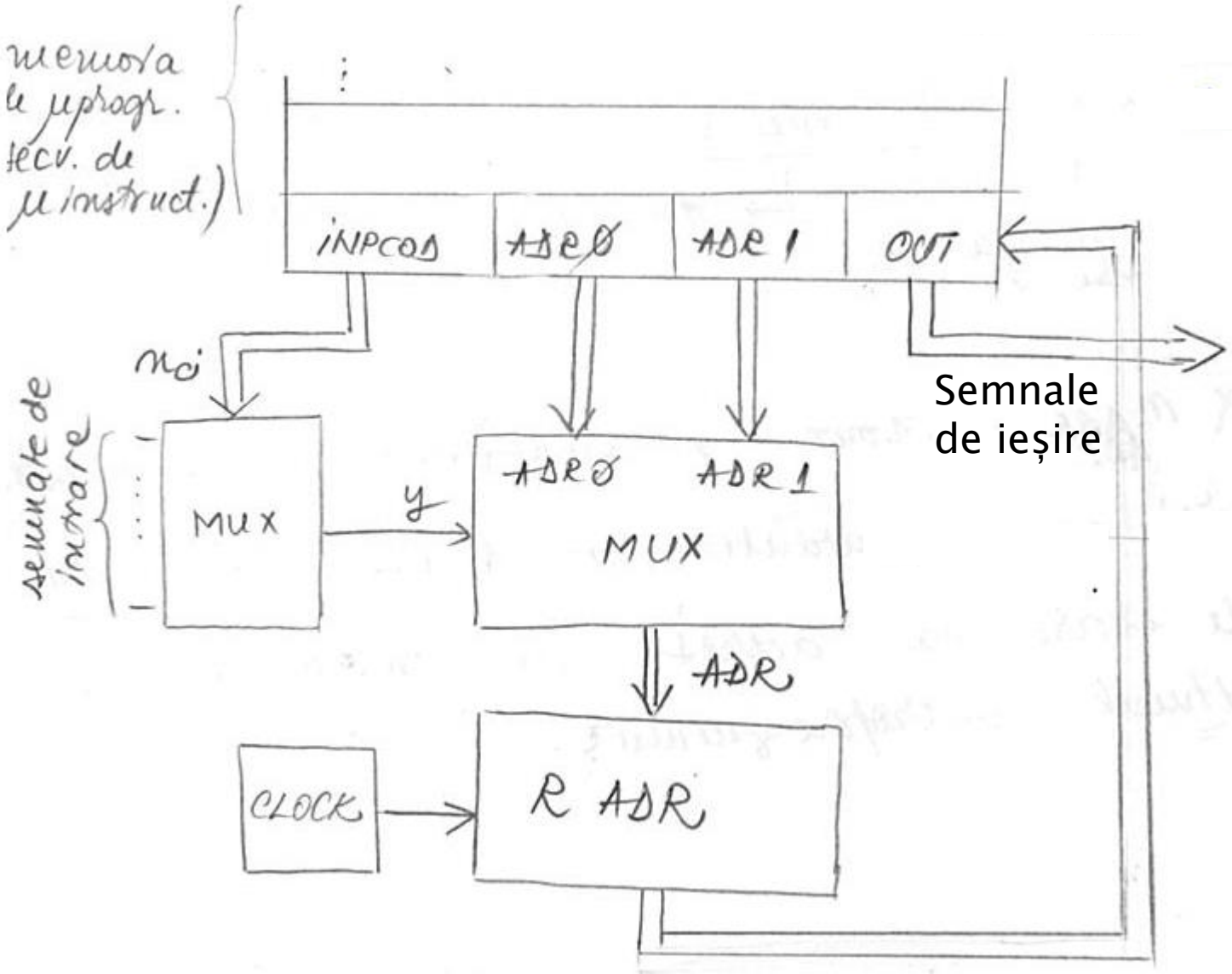
$8 \cdot 7 = 56 > 55$

Problemă (6)

- Alt exemplu: memorii 1K * 8 → avem 1024 stări → RA are 10 biți
- $L_{\mu i} = 6 + 2 * 10 + 25 = 51$ biți
- Avem nevoie tot de 7 circuite, 5 biți din memoria corespunzătoare MSB rămânând nefolosiți
- Alt exemplu: 1024 stări, dar memoria 4K * 8 → $L_{\mu i} = 55$ biți → 7 circuite, 1 bit nefolosit în memoria corespunzătoare MSB și ¾ din fiecare din cele 7 circuite de memorie rămane nefolosită

Structura de comandă microprogramată (1)

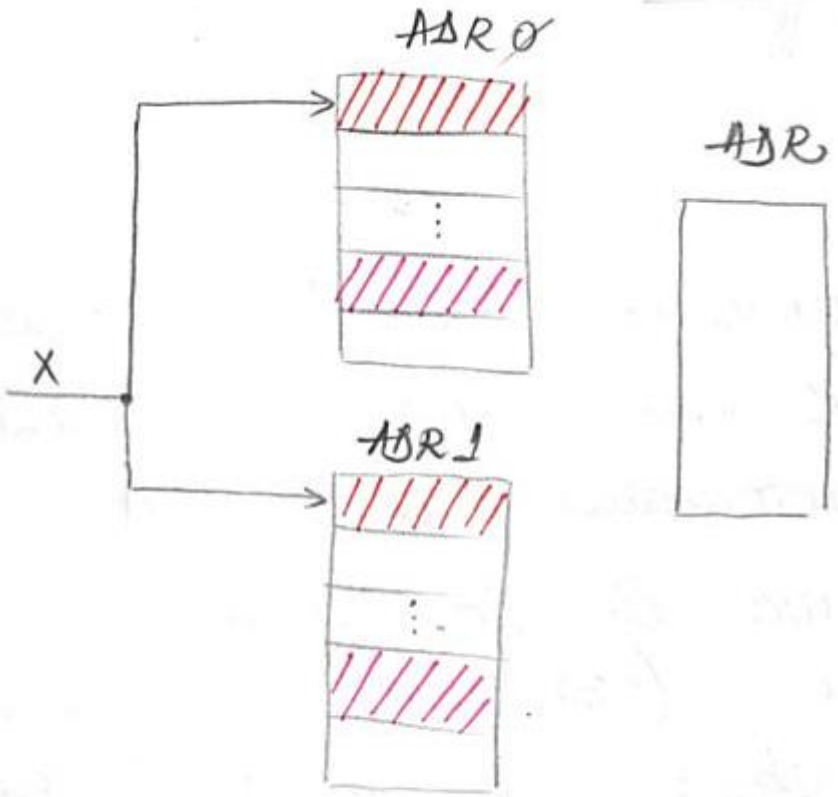
- Până acum am văzut cum se organizează memoria de microprogram → am văzut unde se află informația.
- Acum vom vedea ce informație punem în fiecare locație de memorie pentru a se înlănțui corect stările și pentru a scoate corect ieșirile

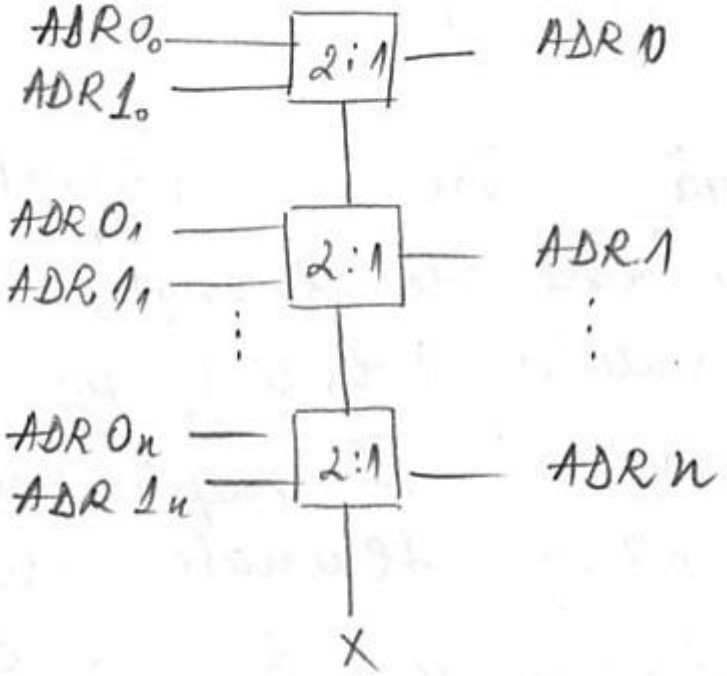


Structura de comandă microprogramată (2)

- INPCOD reprezintă codul de selecție pentru MUX (îmi va selecta una dintre intrări)
- Pe intrările acestuia se vor pune intrările circuitului (semnalele de intrare): I_0 ... I_n (în cazul nostru I_0 ... I_{36})
- Ieșirea MUX va reprezenta valoarea uneia dintre decizii (→ cea care decide dacă mă duc la ADR0 sau la ADR1) și intră într-o structură mai specială MUX 2:1. Această structură este de fapt o baterie de MUX, dar care nu sunt cuplate în cascadă

Bateria de MUX





- Pt X = 0 se obțin $ADR0_0$, $ADR0_1$ ... $ADR0_n$
- Pt X = 1 se obțin $ADR1_0$, $ADR1_1$ ... $ADR1_n$ } pachete de MUX
- Concatenând acești biți se obține următoarea adresă

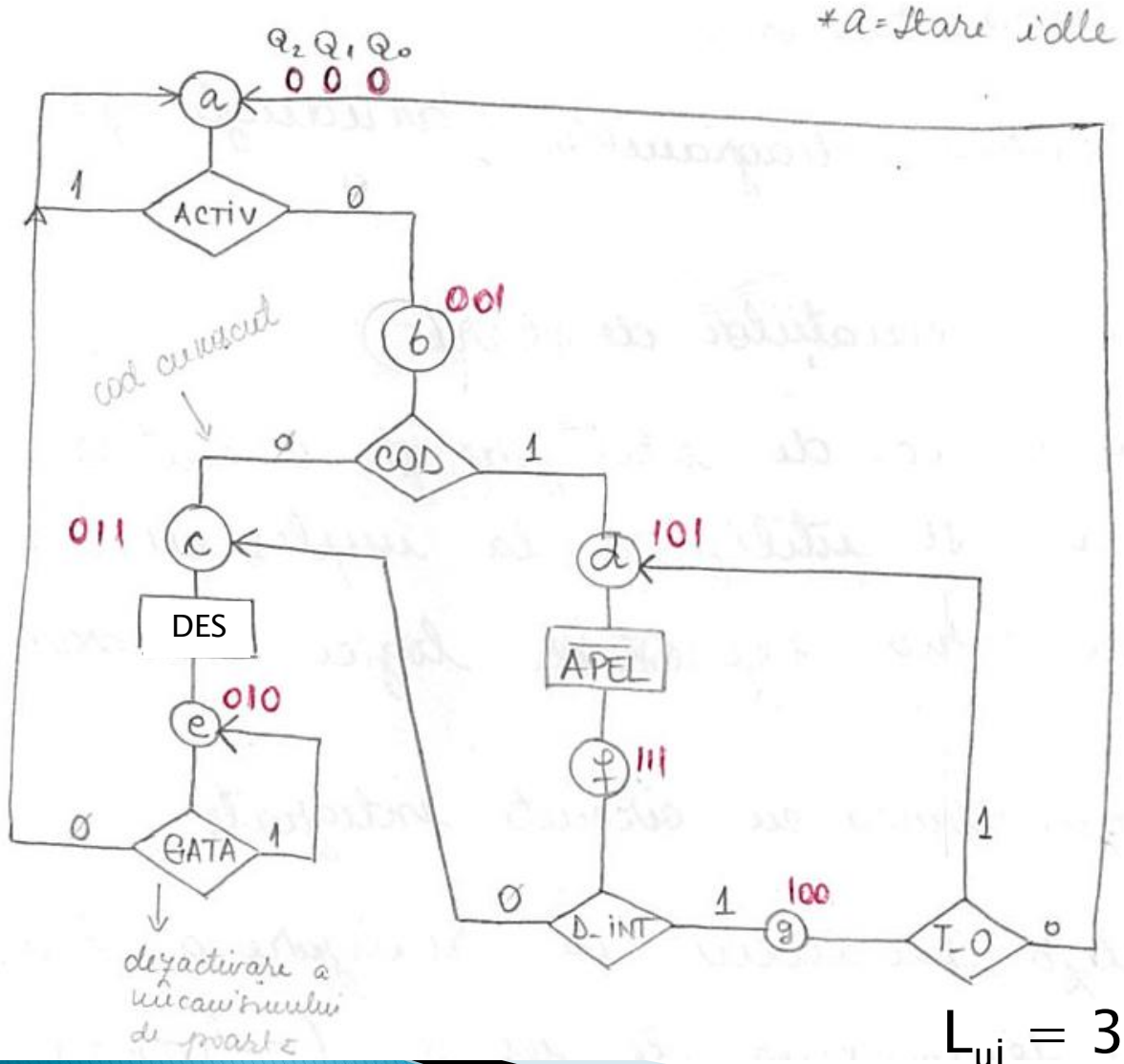
Structura de comandă microprogramată (3)

- În microprogram avem o succesiune de microinstrucțiuni
- La un moment dat, se activează o microinstrucțiune
- Din câmpul INPCOD se selectează una dintre intrări, aceasta putând fi 0 sau 1
- Mai departe, bateria de MUX generează în RADR adresa corespunzătoare din microinstrucțiune și microprogramul continuă cu acea adresă
- Așa se trece prin întregul microprogram
- Dacă am prea multe intrări, MUX inițial poate fi de fapt o cascadă de MUX-uri astfel încât să se acopere toate intrările

Examen

- Dacă vi se cere să implementați cu structuri microprogramate, trebuie neapărat să:
- Calculați lungimea microinstrucțiunii
- Completați memoria microprogramului:
  - alegeți tipul de memorie (e posibil să vi se ceară explicit un anumit tip de memorie)
  - să spuneți câte circuite de memorie sunt necesare

Problema cu interfonul



- Câte intrări?
- 5
- Câți biți?
- 3
- Care sunt intrările?
- ACTIV, COD, GATA, D_INT, T_O
- Câte stări?
- 7
- Câți biți?
- 3
- Câte ieșiri?
- 2 (!APEL, !DES)
- Cât e $L_{\mu i}$?

$L_{\mu i} = 3 + 2 * 3 + 2 = 11$ biți

Problema cu interfonul – memoria de microprogram (1)

- Am făcut o modificare: am pus DES și !APEL ca să vedem cum sunt tratate atât ieșirile active pe 0 cât și cele active pe 1
- Trebuie să completăm tabelul:

- Atenție! Codificarea stărilor nu are nicio legătură cu numărul cuvântului din memorie.
- Se poate folosi același cod în memorie ca în organigramă, dar acest lucru nu e obligatoriu
- Noi vom folosi aceeași codificare

	INPCOD	ADR0	ADR1	DES	!APEL
000					
001					
010					
011					
100					
101					
110					
111					

Problema cu interfonul – memoria de microprogram (2)

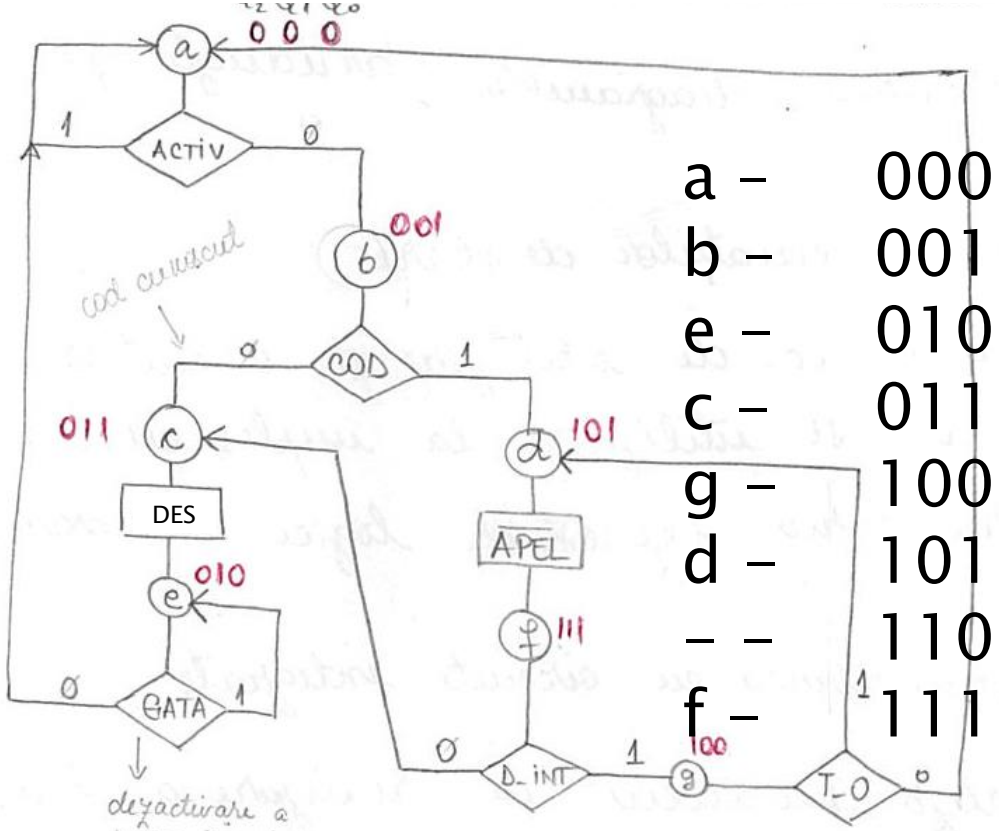
- Primul lucru: trebuie să codificăm intrările → când facem tabelul, trebuie specificat codul fiecărei intrări
- ACTIV – 001, COD – 010, GATA – 011, D-INT – 100, T_0 –101
- La ieșiri: în 000 nu se generează ieșiri → DES (activ pe 1) rămâne în 0 în timp ce !APEL (activ pe 0) rămâne în 1

a –

	INPCOD	ADR0	ADR1	DES	!APEL
000	001	001	000	0	1
001					
010					
011					
100					
101					
110					
111					

Problema cu interfonul – memoria de microprogram (3)

ACTIV – 001, COD – 010, GATA – 011, D-INT – 100, T_0 –101



a – 000
b – 001
e – 010
c – 011
g – 100
d – 101
– – 110
f – 111

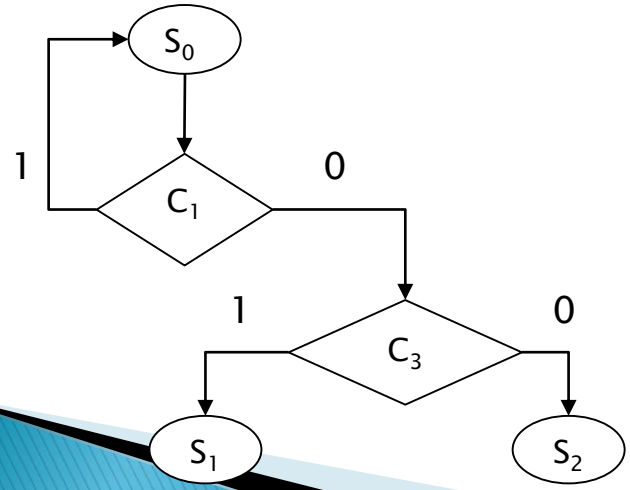
INPCOD	ADR0	ADR1	DES	!APEL
001	001	000	0	1
010	011	101	0	1
011	000	010	0	1
000	010	010	1	1
101	000	101	0	1
000	111	111	0	0
xxx	xxx	xxx	x	x
100	011	100	0	1

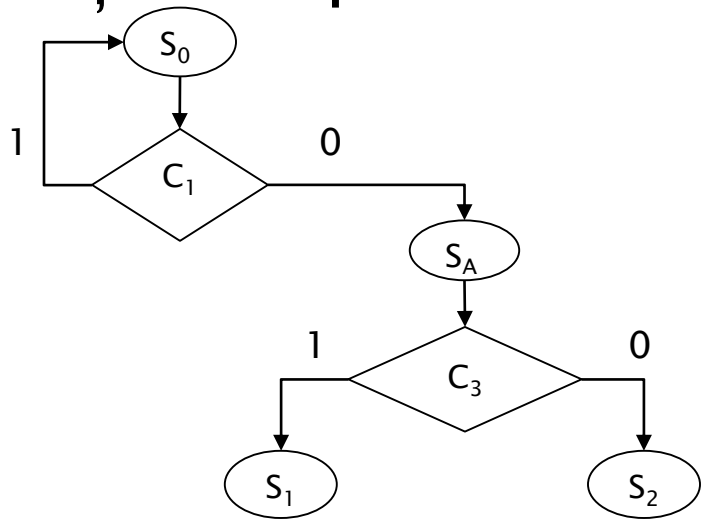
- Nu rămân mereu coduri nefolosite (intrări = $2^{\text{biți}}$), iar dacă nu rămân, punem un cod oarecare
- La salturile necondiționate se pune același lucru pe ADR0 și ADR1

- Ce se întâmplă în C?
- Putem pune XXX sau, dacă rămân coduri nefolosite, pun 000 → de asta începem codificarea de la 001
- 000 va fi codul pentru salturile necondiționate

Examen

- Mai pot apare probleme de genul
- Cum rezolvăm?
- Trebuie să adăugăm stări suplimentare (S_A) pentru că în MUX-ul inițial nu se generează semnal decât de pe o singură intrare și de aceea nu pot avea condiții compuse





Dezavantaj microinstrucțiuni cu format fix

- Lungimea microinstrucțiunii poate să devină prea mare
  - Interfon: 7 stări, $L_{\mu i} = 11$ → dacă avem circuite de memorare pe 4 biți avem nevoie de 3, dacă sunt pe 8 biți avem nevoie de 2
- Data viitoare vedem cum se poate scrie microinstrucțiunea pe mai puțini biți
- Observație legată de ieșiri:
  - În exemplul nostru, informația utilă apare doar in 2 locuri: când !APEL devine 0 și când DES devine 1
  - În rest, este informație irosită
  - Vom vedea cum în loc să punem această informație, punem altceva astfel încât să se micșoreze lungimea microinstrucțiunii

ÎNTREBĂRI?